

构建可扩展的AI Agent

从算力到数据的架构实践

郑予彬

资深开发者布道师 | 亚马逊云科技



Content 目录

01 AI正在从工具变成同事

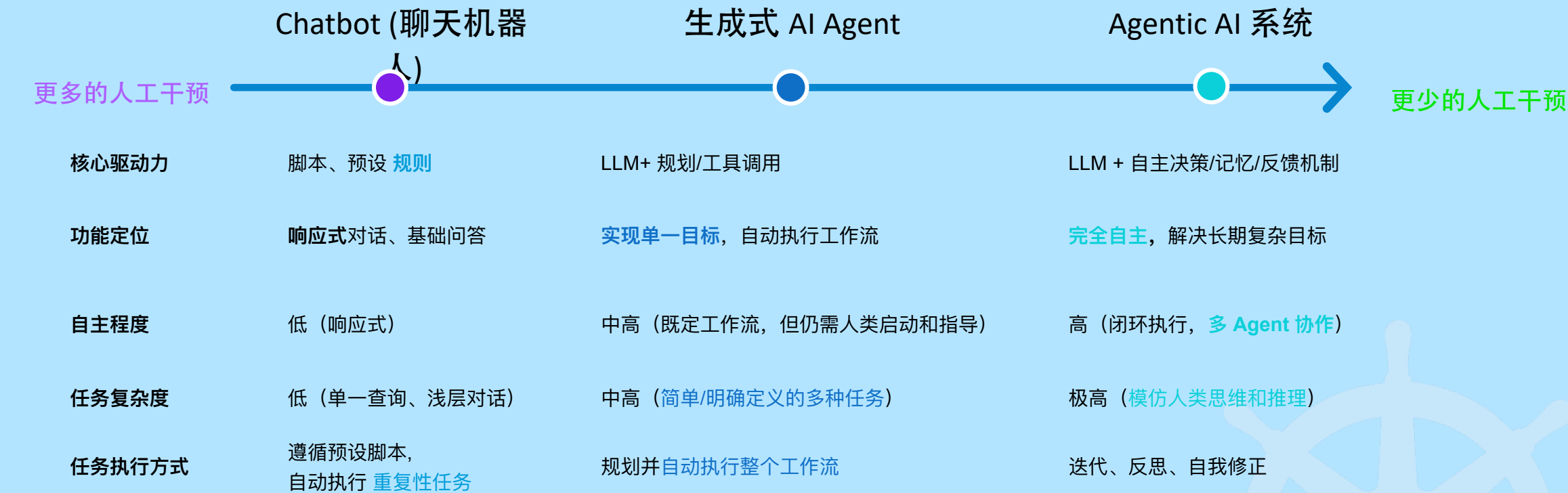
02 Agent的演进

03 构建可扩展的AI Agent

04 AI Agent 构建中的实践



Agentic AI 的发展历程



Predictive AI

Generative AI

Agentic AI



AI正在从工具变成同事

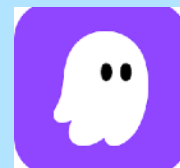


越来越多的企业正在把AI接入业务流程

《Four Futures for Jobs in the New Economy: AI and Talent in 2030》 Published on 7 January 2026

2022 → 55%企业使用AI

2025 → 88%企业使用AI



我的AI同事们

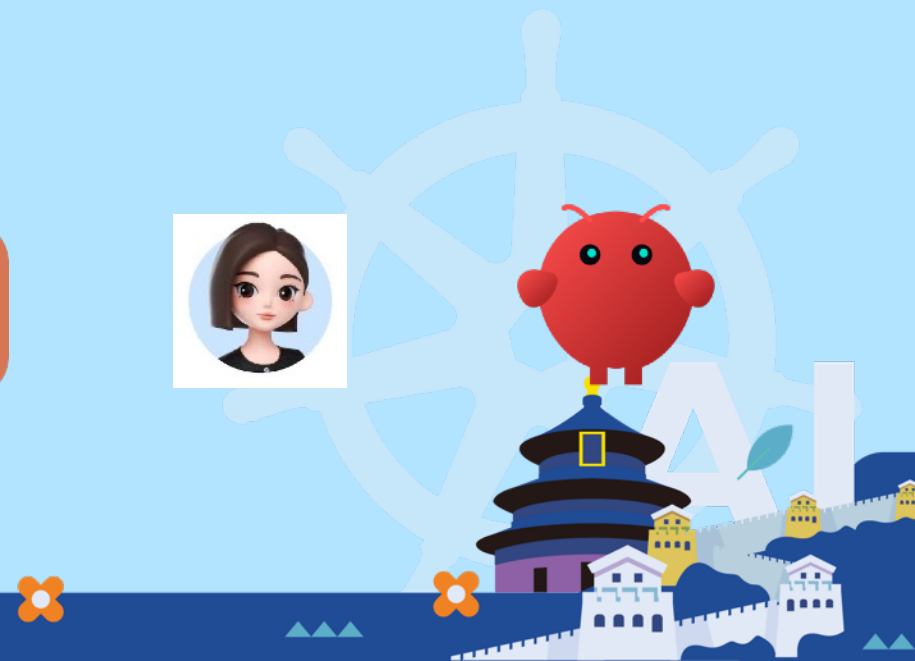
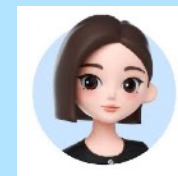
AI正从“回答问题”走向“参与任务”

《Which Economic Tasks are Performed with AI?》

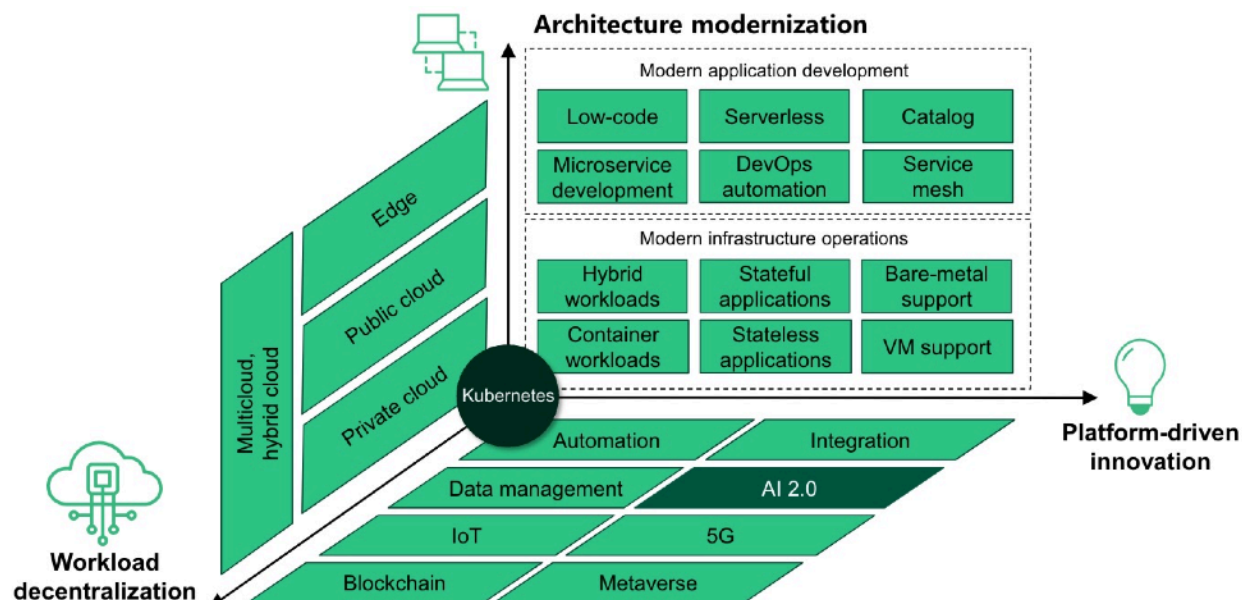
Anthropic 分析了数百万次 Claude 对话，发现一个很有意思的结果，AI 使用模式为：

能力增强(Augmentation) 57%

任务自动化(Automation) 43%



Agent 成为新的应用形态，云原生成为默认底座



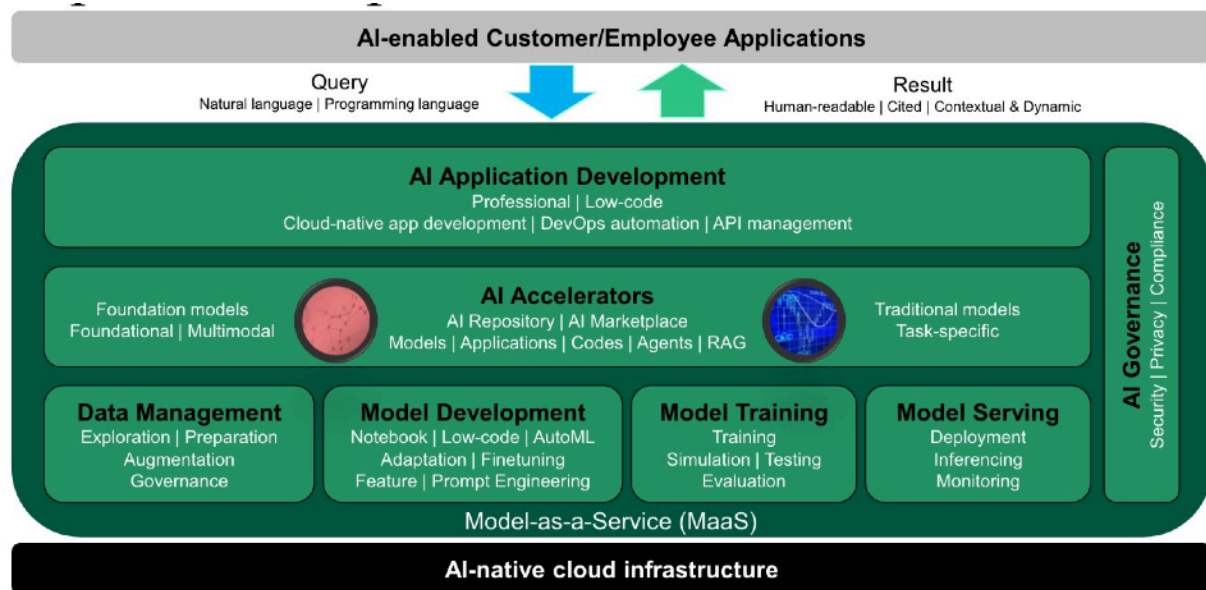
Agent 像一个持续运行的系统：

- 长生命周期的（long-running）— Agent 不再是一次请求-响应，而是持续存在、持续交互
- 弹性伸缩（elastic scaling）— 随着用户请求和任务变化，动态扩展计算资源
- 工具与服务编排 (tool orchestration) — 需要调用API、数据库、外部系统等多种能力
- 分布式执行 (distributed execution) — 多组件协同，包括模型、工具、存储和调度系统

原生架构擅长解决的核心问题

- 👉 计算的弹性
- 👉 服务的解耦
- 👉 系统的可扩展性
- 👉 以及大规模分布式运行能力

Agent 的问题，正在从“模型能力问题”，转向“系统架构问题”：



算力，决定规模边界

PROJECT RAINIER



算力，决定规模边界

100万+ 枚

TRAINIUM 芯片
已部署

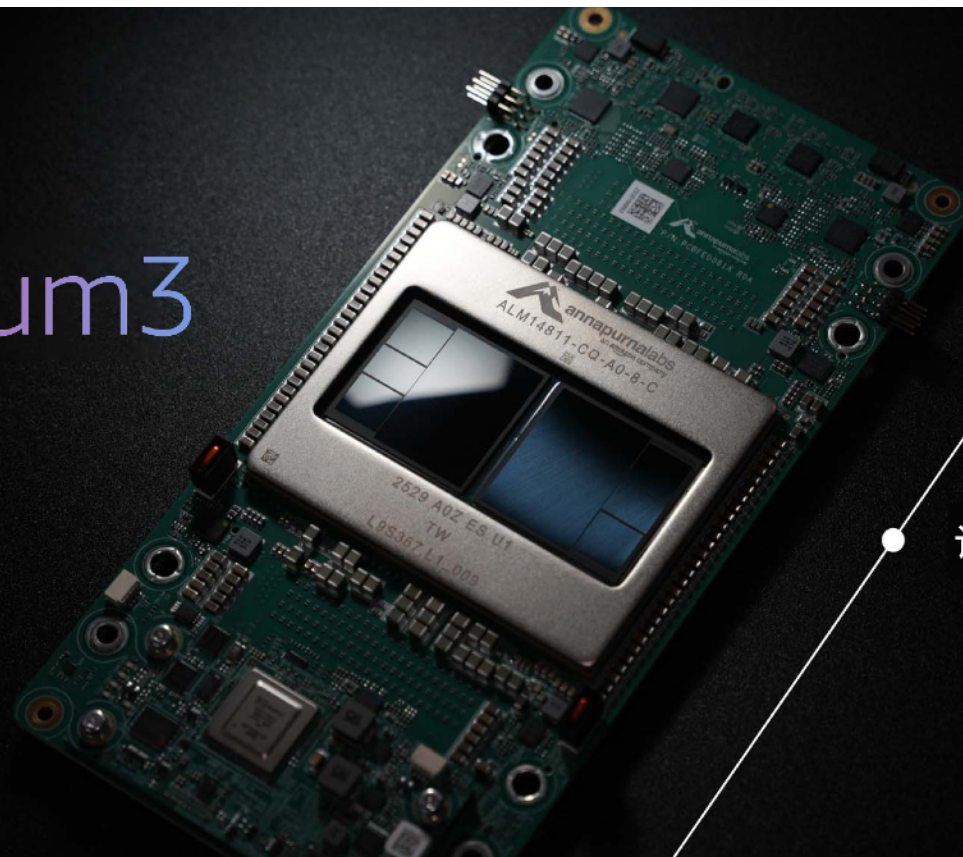


算力，决定规模边界

Amazon Trainium3

专为新一代生成式 AI
工作负载打造的芯片

PyTorch native



3 纳米工艺节点

40% 能效提升

计算能力提升 2 倍

没有低成本推理，就没有规模化 Agent



算力，决定规模边界

云为NVIDIA GPU 提供了更理想的运行环境

优异的 可靠性 和 可用性 (面向基于 GPU 的系统)

P6e

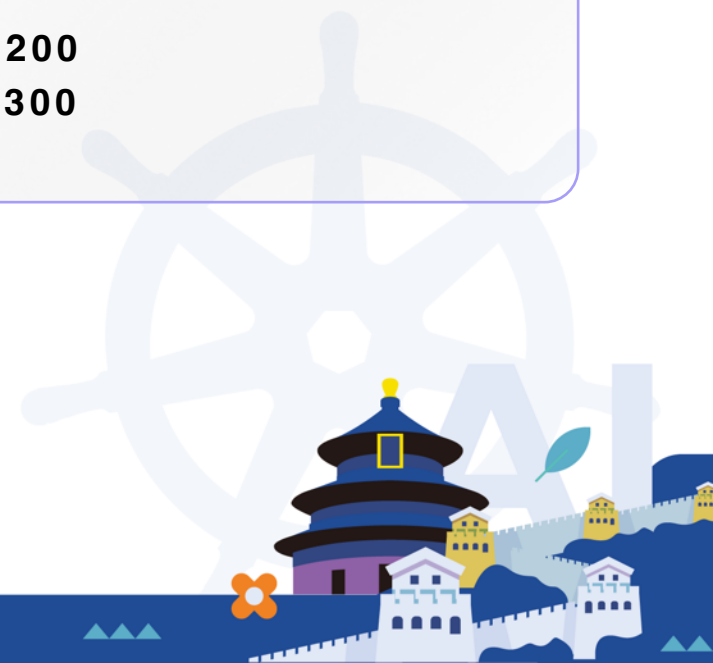
GB200 [NVL72]

GB300 [NVL72]

P6

B200

B300



数据，决定智能边界

数据，本质上是 Agent 的记忆



语义数据（理解）

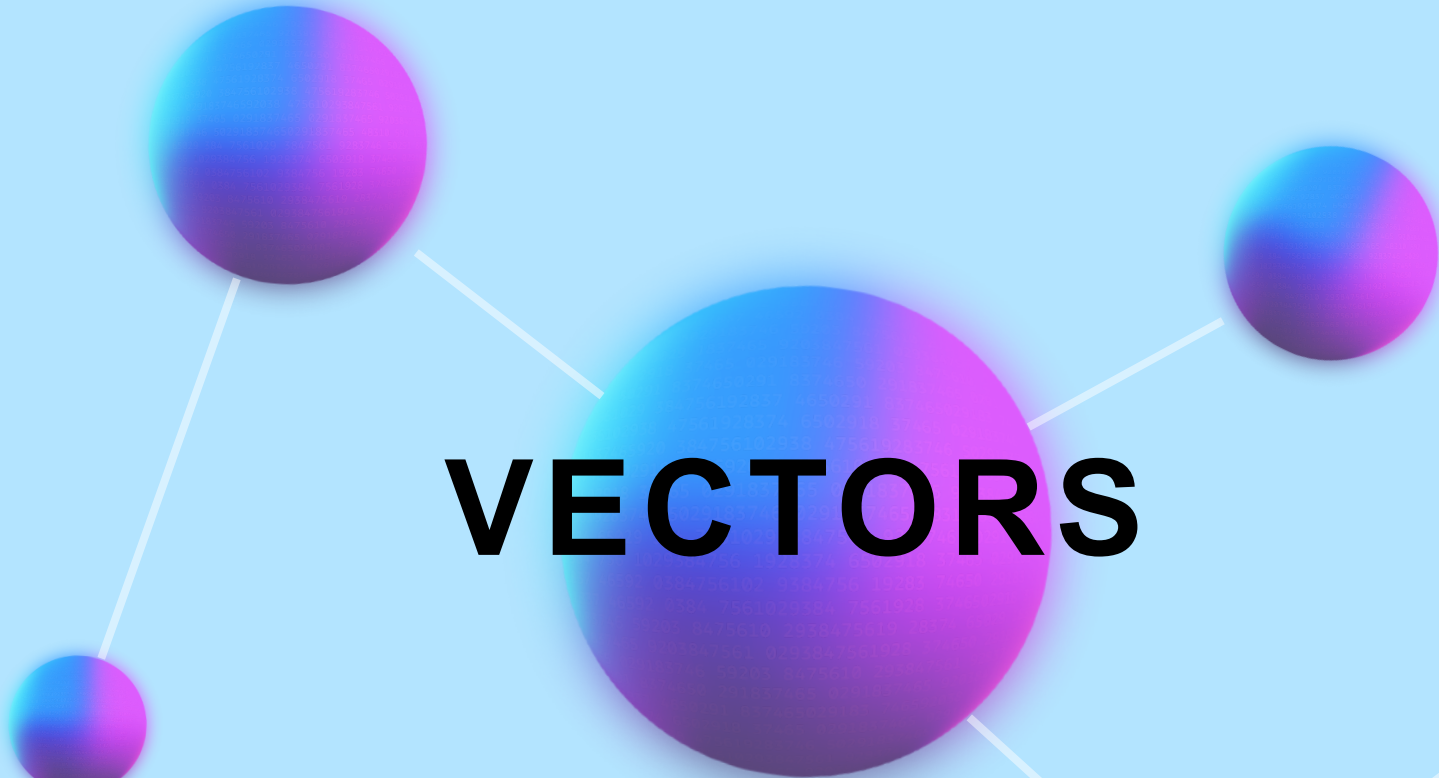


向量数据（相似性）



结构化数据（规则）

数据，决定智能边界



VECTORS

这是搜索、RAG 和 Agent 的基础



Amazon
OpenSearch
Service GPU
acceleration

十亿级

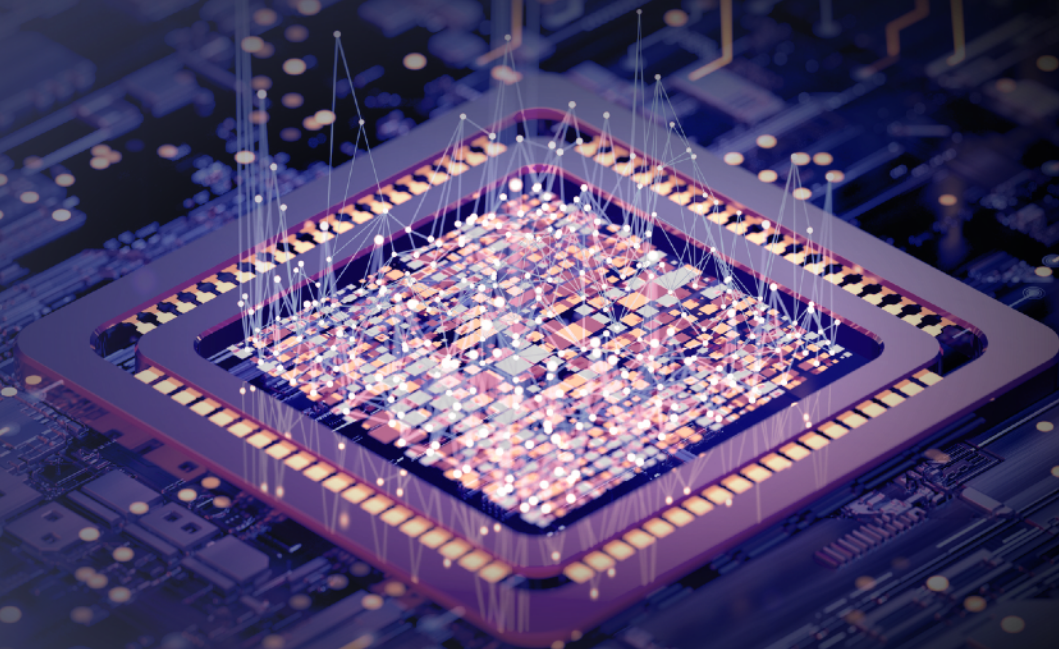
向量数据库构建时间

<1 小时

10X

向量检索速度，同时成本降低至

25%



数十亿向量

向量规模爆发，存储架构的演进 语义层

提供十亿级向量存储，可使上传、存储和检索向量
成本减少高达 90%



<100ms

专为热查询设计，
可使时延低至 100 ms

20亿

每个索引最多可支持存储和查询 20
亿个向量——相比预览版提升 40 倍

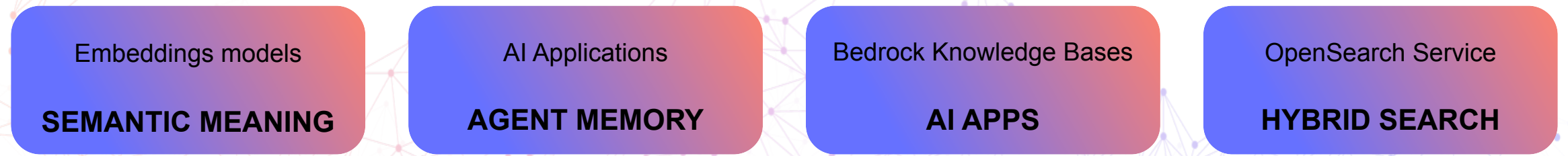
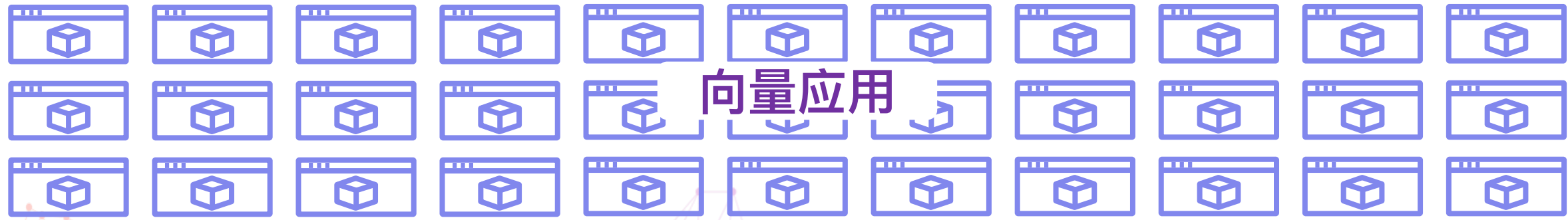
10,000

每个存储桶可包含 1 万个索
引，总计可支持高达 20 万亿个
向量

1,000

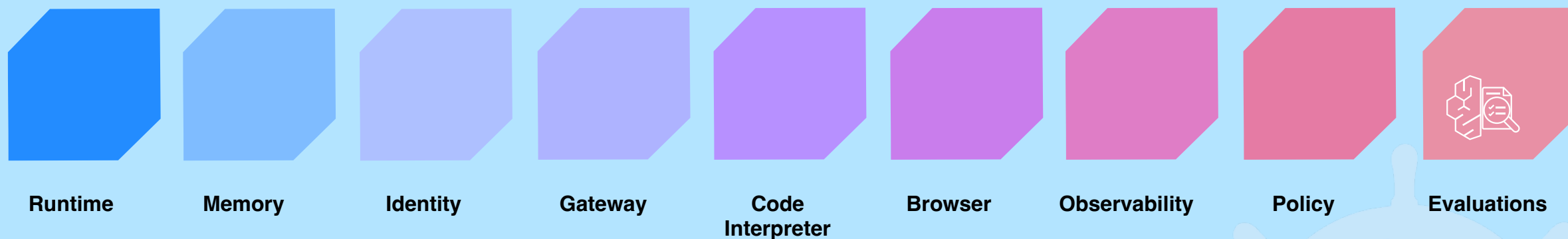
流式单向量更新速率可达 1000 个向量/
秒，
每次最多可返回 100 条检索结果

向量应用



系统，决定如何运行

让Agent真正运行起来的系统

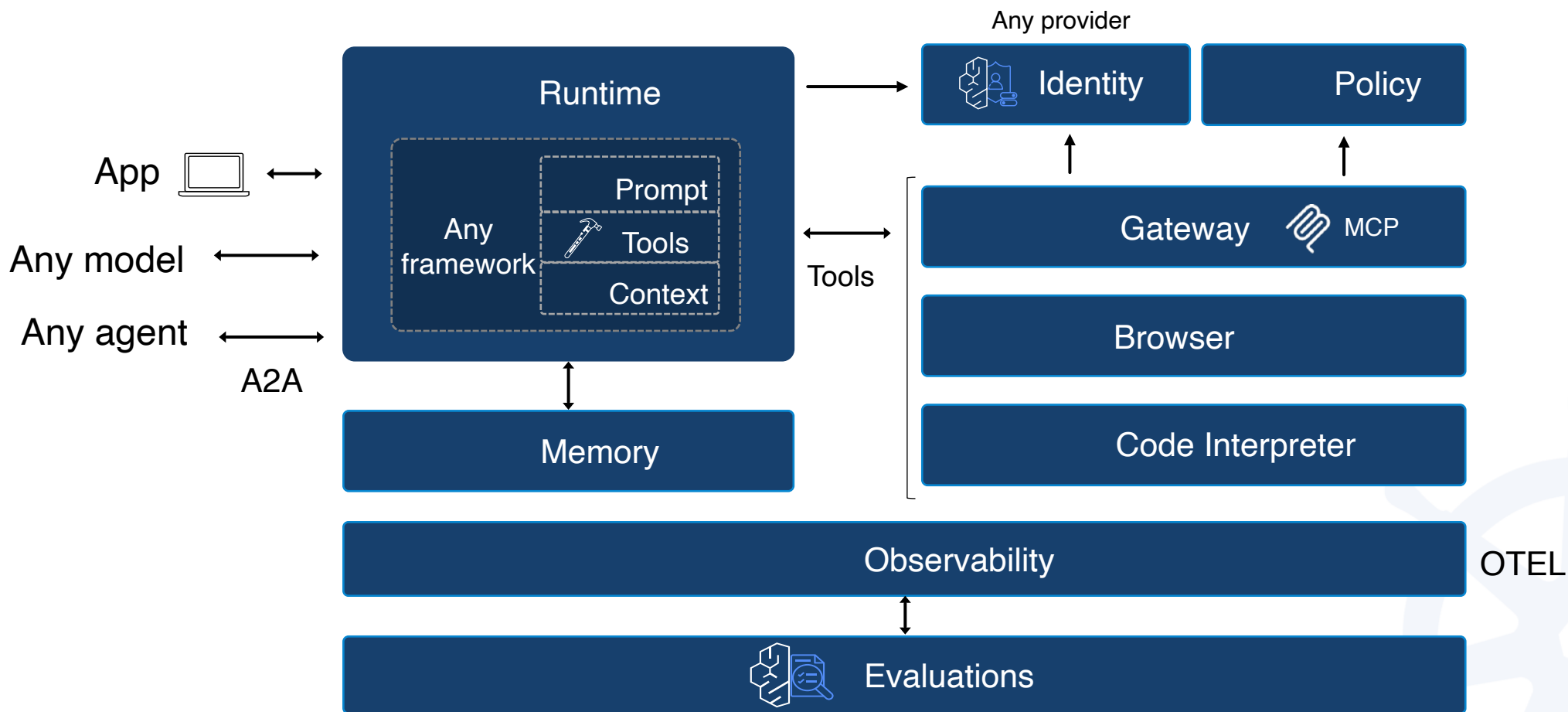


Agent $\neq$ function Agent = runtime system



系统，决定如何运行

系统，不只是运行，更关键是协同



Execution (运行) | State (状态) | Control (控制)



OpenClaw

Multi-Channel AI Agent Gateway

Security

Compute/Container

Networking

Database/Storage

Architecture

消息通道层

WhatsApp · Telegram · Discord · iMessage · Slack · Mattermost · Feishu

网关核心层

WebSocket Server · Routing · Session · Queue · Prompt · Tool Execution

Agent Runtime

Agent Loop · Inference · Tool Streaming · Compaction · Memory Flush

Model & Storage (Memory)

Anthropic · OpenAI · Bedrock · Gemini · Local LLM · SQLite · JSONL Transcripts



维度1：安全隔离同时兼顾按需伸缩

Architecture View



→ 每个用户一个独立沙箱，Serverless 自动伸缩

Firecracker microVM



A

A的龙虾会话

独立 CPU / 内存 / 文件系统



完全隔离

Firecracker microVM



B

B的龙虾会话

独立 CPU / 内存 / 文件系统



完全隔离



按需自动伸缩
Serverless



E2E冷启动：5s



最长运行：8 小时



会话结束：自动清理，零残留



隔离技术：Firecracker



用户隔离



会话隔离

传统容器方案 (路径3)

- ✗ 共享内核：仅依赖 Namespace/Cgroup，攻击面大
- ✗ 软隔离：存在容器逃逸风险(CVE-2024-21626)
- ▲ 数据残留：销毁不彻底，本地卷风险

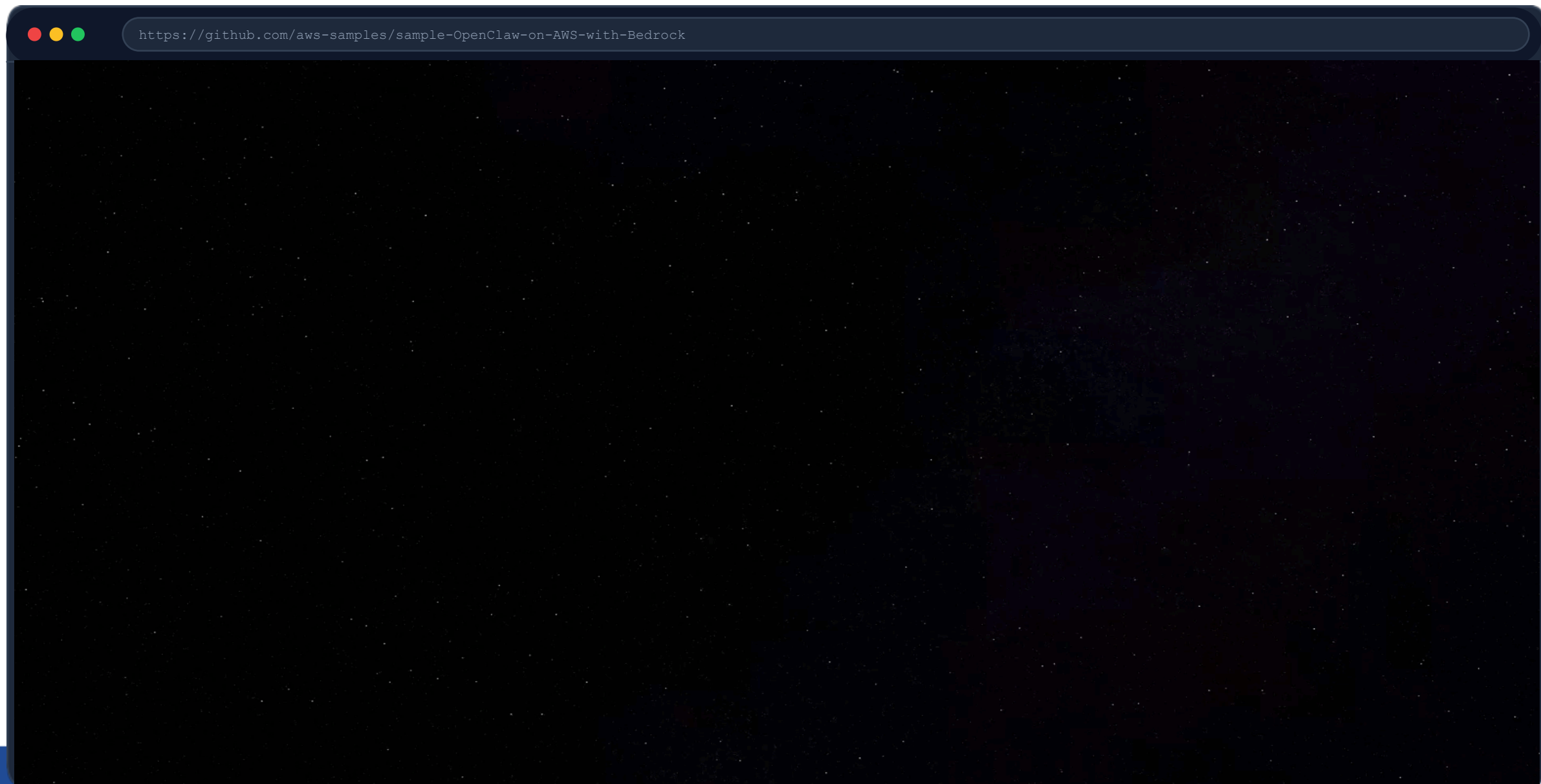
AgentCore

Firecracker microVM (路径4)

- ✓ 硬件级虚拟化：基于 KVM，独占 CPU/Mem
- ✓ 强隔离：攻击者无法跨越 VM 边界
- ✓ 阅后即焚：会话结束毫秒级销毁

Demo: OpenClaw 企业级隔离

两只 OpenClaw 无法查看对方的密钥



维度2：企业治理 – 快速实现预设的企业规则

自建

数十周

Policy Engine
控制 Agent/用户 调用的工具与参数

2-4 周开发 + 维护

Identity 集成
对接 IdP (Okta/AD) 实现 SSO

2-3 周开发 + 测试

API Gateway
拦截工具调用, 限流 + 认证 + MCP

3-6 周开发

审计系统
全链路日志采集 + 存储 + 查询

2-3 周开发

凭据管理
集成 Secrets Manager + 轮换

1-2 周开发

合计 5 个子系统
1-2 名工程师全职投入

VS

全托管 (AgentCore)

数小时

✓ **内置 Policy**
Cedar 声明式规则, 无需编码

✓ No Code

✓ **原生 Identity**
无缝集成 IdP, SSO 开箱即用

✓ Config

✓ **托管 Gateway**
全链路拦截 + 协议自动转换

✓ Built-in

✓ **原生可观测性**
全链路 Trace 开箱即用

✓ Native

✓ **安全凭据**
Secrets Manager 集成

✓ Secure

全部内置, 天级配置
非月级开发

自建方案

2.5 - 4.5 个月全职工程投入

AgentCore 方案

小时级配置, 开箱即用



场景深入：防止数据与企业资产的越权访问



大型企业环境

200+ Agent 服务不同部门



调用者：

市场部 (Marketing) 员工的龙虾 Agent



风险操作：

调用 财务系统 API 获取预算



核心隐患：

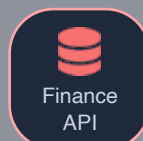
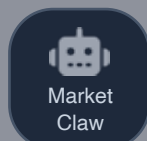
Agent 默认继承全量权限，无边界

// AgentCore Cedar 策略示例

```
forbid ( principal in Group::"Marketing", action ==  
Action::"GetBudget", resource == Tool::"FinanceAPI" );
```

不做企业治理

隐式信任



调用成功：数据泄露

事后审计才发现
无法阻止实时访问

做企业治理 (AgentCore Policy)

防护流程



1. Identity 身份识别

系统识别为 "Marketing Claw User"



2. Policy 规则匹配

策略：市场部 禁止访问财务工具



3. Gateway 实时拦截

请求阻断，无法访问



4. Observability 审计

自动记录拦截日志，触发告警

Demo: OpenClaw 企业权限管控

通过 AgentCore Policy 实现 OpenClaw 企业级权限管控

LLM



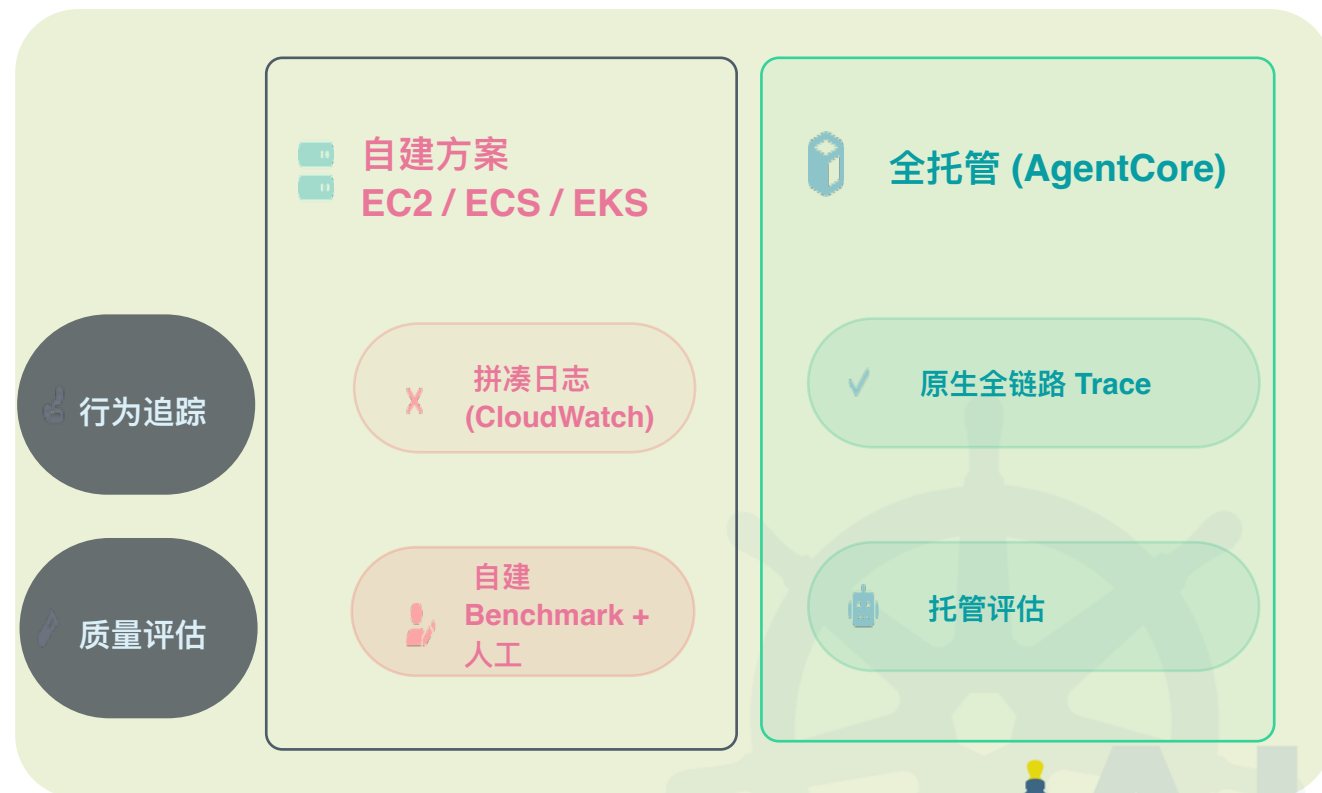
<https://github.com/aws-samples/sample-OpenClaw-on-AWS-with-Bedrock>



维度3：可观测性 – 随时监控您龙虾平台的状态

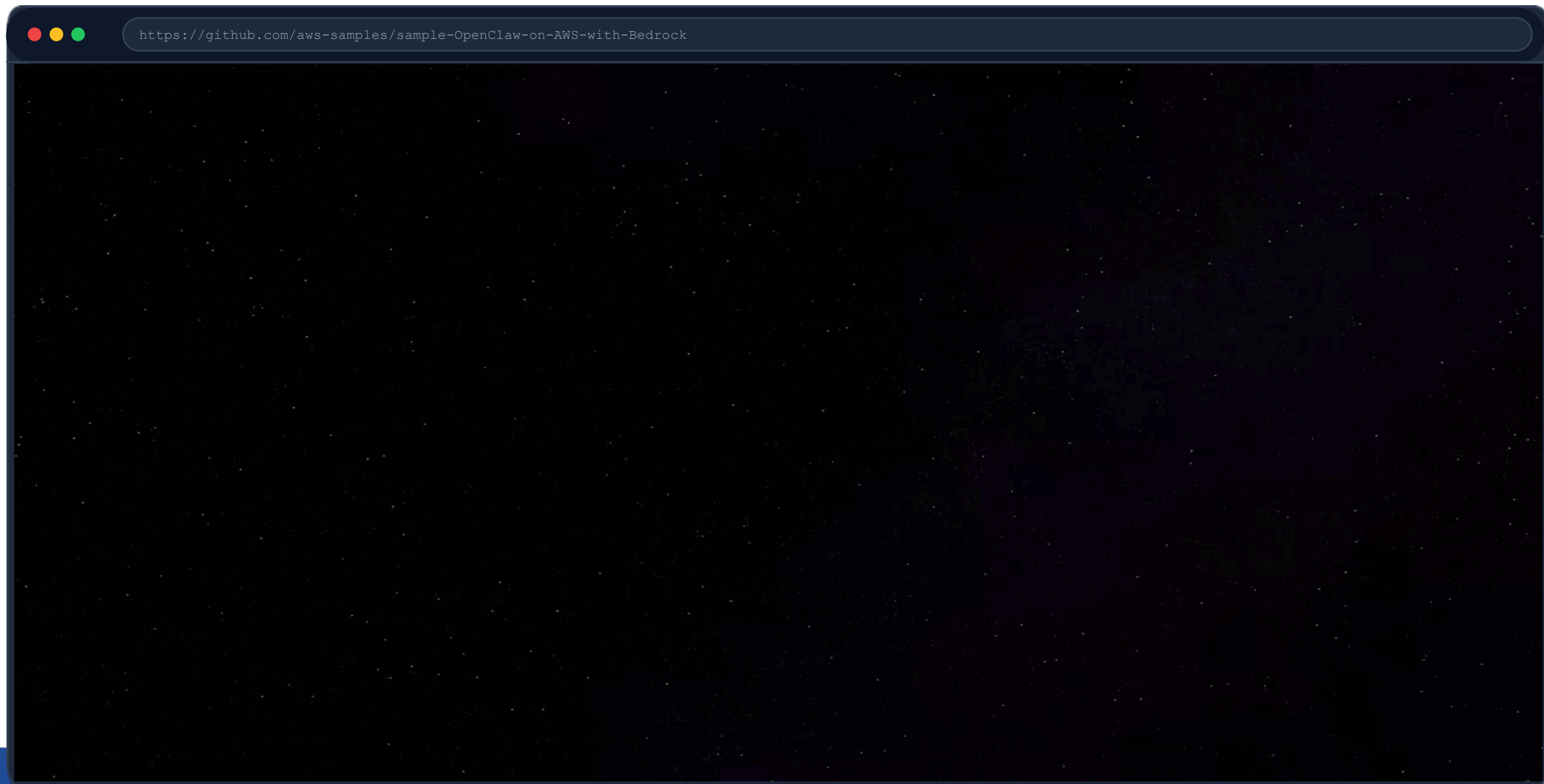
超过 53% 企业用户在短短一个周末内就给 OpenClaw 授予了特权访问权限。问题不只是“装得快”，而是企业通常缺乏配套的治理、审计与运行时监控能力，难以验证 Agent 是否始终在安全、合规、可控的边界内运行。

[Noma Security 数据](#), Feb 9th



Demo: OpenClaw 全链路追踪

在控制台追踪 OpenClaw 的可观测性指标



维度4：成本压力 – 一个龙虾还好，成千上万个呢？

基础设施节省分析

"场景越碎片化、闲置时间越长，按需付费的优势越明显。"

场景 1 (高频)

54%

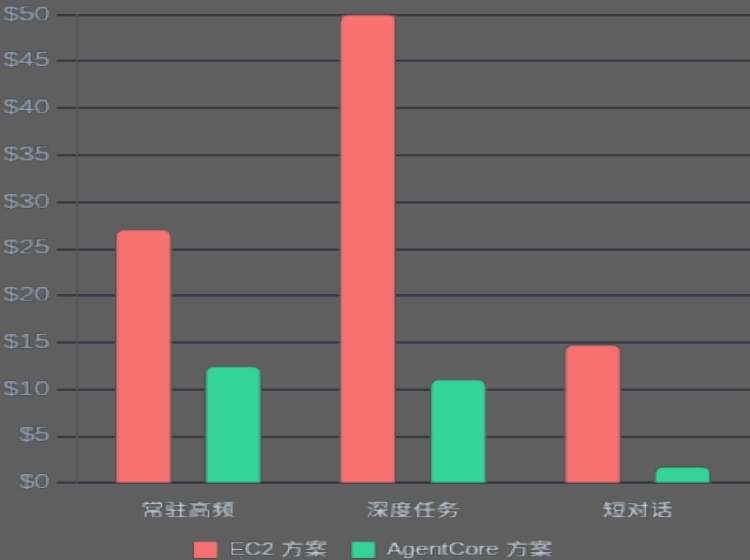
场景 2 (深度)

78%

场景 3 (短对话)

88%

单人月度基础设施成本 (\$)



Up to 88%
(基于上述假设)

场景 1：7×24 常驻助手，高频使用				节省 54%
假设：100次/天, 30天, t4g.medium常驻 vs Agent按需				
项目 (10/100/500人)	10人	100人	500人	
EC2 (t4g.medium \$25+EBS \$2) 7×24 常驻 (730h/月)	\$270	\$2,700	\$13,500	
AgentCore Runtime CPU \$6.7 + Memory \$5.7 (按需)	\$124	\$1,240	\$6,200	
基础设施节省	54%	54%	54%	

场景 2：工作日深度任务，长时间 session				节省 78%
假设：8h session (活跃4h), 22天, t4g.large vs Agent按需				
项目 (10/100/500人)	10人	100人	500人	
EC2 (t4g.large \$48+EBS \$2) 支撑长时间复杂任务	\$500	\$5,000	\$25,000	
AgentCore Runtime CPU \$7.9 + Memory \$3.3	\$110	\$1,100	\$5,500	
基础设施节省	78%	78%	78%	

场景 3：个人助手，工作时间短对话				节省 88%
假设：100次/天 (60s session), 22天, t4g.small常驻 vs Agent按需				
项目 (10/100/500人)	10人	100人	500人	
EC2 (t4g.small \$12+EBS \$2.4) 7×24 常驻	\$147	\$1,466	\$7,330	
AgentCore Runtime CPU \$1.0/人 + Memory \$0.7/人	~\$17	~\$170	~\$850	
基础设施节省	88%	88%	88%	

注：比较不包含模型成本比较，取决于选择的模型；辅助服务 (S3/DTO/etc)，两方案均需要。

云上托管是高性价比和强安全性的优选择

- 隔离性
- 权限治理与安全性
- 可观测性
- 基础设施成本
- 运维难度
- 持久化存储

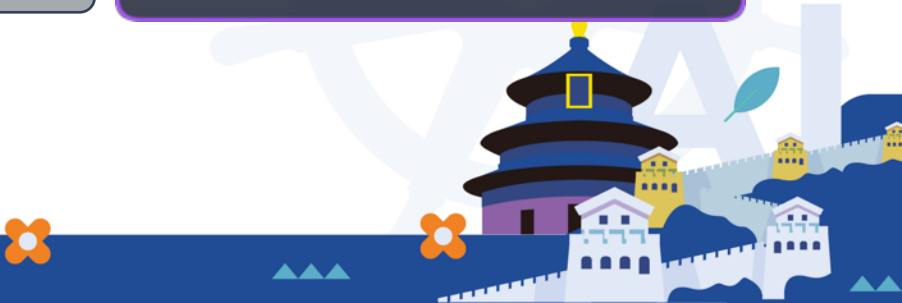
EC2
VM 级隔离
较强
全部自建
较贵
较高
✓原生支持

ECS / EKS
容器级
较强
全部自建
较贵
较高
✓原生支持

最佳选择

AgentCore

- ✓ microVM 硬件级
- ✓ 最强
- ✓ 原生支持的 Agent 全链路观测性
- ✓ 最便宜, I/O 等待不收费
- ✓ 较低
- ⌚ 自建 S3 持久化存储方案, 原生持久化存储 (Coming)



Demo: OpenClaw 端到端体验

用户登录开始使用 OpenClaw on AgentCore 方案 01:06



<https://github.com/aws-samples/sample-OpenClaw-on-AWS-with-Bedrock>



构建可扩展的AI Agent - 本质是系统工程

- 计算，决定规模
- 数据，决定智能
- 系统，决定运行

从算力，到数据，到系统，构成了 Agent 架构的完整基础



Thanks

